memcpy 性能分析及优化手段

CH/SR 都使用了自定义的 memcpy函数来提升memcpy性能

1. 避免从动态库调用函数，避免PLT calls(Procedure Linkage Table，过程链接表调用)以被编译器inline
2. 规避glibc兼容性问题
3. 根据数据源长度，针对性实现更高效的拷贝策略，提升性能

基本思路

- use overlapped move to optimize the case when size <= 16
- use avx(256bit reg) move to optimize the case when size <= 256，CH此策略阈值是128，比SR偏小
- use erms(enhanced repeat movsb/stosb) to optimize the case when size in [512KB,2MB] 此项为SR的再优化，使用汇编指令显式调用erms，而不是像CH一样依赖编译器优化，目的是使实际指令受编译器影响更小
- use unrolled avx(256bit reg) move to optimze the rest cases.

CH：
代码中注释 Custom memcpy implementation for ClickHouse.
思路参考自 lz4，分析文章：https://habr.com/en/companies/yandex/articles/457612/

SR：
SR对CH实现的再实现和分析
https://github.com/StarRocks/starrocks/pull/13330

然而新版本的glibc也通过上述手段以及一些其它手段提升了memcpy的性能，并直接使用汇编实现，各种自定义的memcpy实现或许只剩下在多编译环境上性能稳定的优势(如果环境的glibc版本较低，不包含最新的优化，则性能不如自定义版本)。具体测试后续关注此issue
https://github.com/StarRocks/starrocks/issues/40414

一种开源的memcpy优化实现，已经落后于新版本的glibc
https://github.com/skywind3000/FastMemcpy/issues/6

最新版本glibc的注释

```
/* memmove/memcpy/mempcpy is implemented as:
   1. Use overlapping load and store to avoid branch.
   2. Load all sources into registers and store them together to avoid
      possible address overlap between source and destination.
   3. If size is 8 * VEC_SIZE or less, load all sources into registers
      and store them together.
   4. If address of destination > address of source, backward copy
      4 * VEC_SIZE at a time with unaligned load and aligned store.
      Load the first 4 * VEC and last VEC before the loop and store
      them after the loop to support overlapping addresses.
   5. Otherwise, forward copy 4 * VEC_SIZE at a time with unaligned
      load and aligned store.  Load the last 4 * VEC and first VEC
      before the loop and store them after the loop to support
      overlapping addresses.
   6. On machines with ERMS feature, if size greater than equal or to
      __x86_rep_movsb_threshold and less than
      __x86_rep_movsb_stop_threshold, then REP MOVSB will be used.
   7. If size ≥ __x86_shared_non_temporal_threshold and there is no
      overlap between destination and source, use non-temporal store
      instead of aligned store copying from either 2 or 4 pages at
      once.
   8. For point 7) if size < 16 * __x86_shared_non_temporal_threshold
      and source and destination do not page alias, copy from 2 pages
      at once using non-temporal stores. Page aliasing in this case is
      considered true if destination's page alignment - sources' page
      alignment is less than 8 * VEC_SIZE.
   9. If size ≥ 16 * __x86_shared_non_temporal_threshold or source
```

```
      and destination do page alias copy from 4 pages at once using
      non-temporal stores.  */
```